> Using a python client socket, if the server closes the connection, will my client socket close?

Short answer: No, not automatically.

When the server closes the connection, your client socket **does not close itself**. It remains open (from the OS perspective) until you explicitly call `close()` on it. However, the connection is *half-closed* / broken, and your client will detect this through socket operations.

What actually happens

When the server sends a FIN:

- **Your socket stays open** — `sock.fileno()` still returns a valid descriptor.
- A subsequent `recv()` will return `b''` (empty bytes) **exactly once** — this is the signal that the peer closed its sending side (EOF).
- A subsequent `send()` may succeed the first time (TCP allows sending after receiving FIN — "half-close") but will eventually raise `BrokenPipeError` / `ConnectionResetError`, or the peer's stack may respond with RST.
- If the server sends RST instead of FIN (e.g. it called `close()` with unread data in its buffer, or `SO_LINGER=0`), your next `recv()` raises `ConnectionResetError`, and your `send()` raises `BrokenPipeError` / `ConnectionResetError`.

Important subtlety: "closing" a socket ≠ removing the file descriptor

Calling `sock.close()` on the client just decrements the reference on the underlying fd. The **kernel** only tears down the TCP connection (and reclaims the fd) once the fd is truly closed *and* any lingering conditions resolve. So:

- The fd stays allocated until you `close()` it (or the process exits, or `recv` returns 0 and you close it).
- You can leak file descriptors if you never close on EOF.

Typical client pattern

Typical client pattern

```python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("localhost", 12345))

try:
    while True:
        data = s.recv(4096)
        if not data:            # server closed -> EOF
            print("server closed the connection")
            break               # <- you must break and close
        print("got", data)
finally:
    s.close()                   # <- explicit close
```

Detection options

1. **Blocking** `recv()` → returns `b''` on clean FIN, raises on RST.
2. `select` / `poll` / `epoll` → fd becomes "readable", then `recv()` returns `b''`.
3. `sock.settimeout()` or non-blocking → you get `TimeoutError` / `BlockingIOError` while idle, but EOF still surfaces as `b''`.
4. `sock.getpeername()` may raise `OSError(ENOTCONN)` after RST in some cases.
5. `SO_KEEPALIVE` helps detect *dead* peers (no FIN at all), but not a graceful close.

Bottom line

> The server's `close()` sends a FIN; your client socket object and its fd **remain open** until *you* close it. You must detect EOF (`recv() == b''`) or an error, then call `close()` yourself — otherwise you'll leak descriptors and never free the connection state.

How to wait for the server to close

How to wait for the server to close (in Python)

"Waiting for the server to close" means blocking until your client observes EOF (or a

reset). Here are the reliable ways, from simplest to most robust.

1. Just call `recv()` in a loop until it returns `b''`

This is the canonical way. `recv()` blocks until data arrives, EOF is seen, or an error occurs.

```python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("localhost", 12345))

try:
    while True:
        data = s.recv(4096)
        if not data:               # b'' == clean EOF (server sent FIN)
            print("server closed")
            break
        # process data...
except ConnectionResetError:
    print("server reset the connection")
finally:
    s.close()
```

Key point: `recv()` returning `b''` is *the* signal the peer closed its sending side. It happens exactly once; after that, further `recv()` s return `b''` immediately.

2. If the server sends a lot of data you don't care about

Drain it, otherwise you'll keep looping. Either discard:

```python
while s.recv(65536):
    pass
print("closed")
```

...or just `recv()` once and check — but draining is correct if the server streams before closing.

3. If you're not reading data — `select` / `poll` for readability

EOF makes the socket *readable*. You can wait without consuming data, though you still must `recv()` to distinguish data from EOF:

```python
import select, socket

s = socket.socket()
s.connect(("localhost", 12345))

# block until the socket is readable (data OR EOF OR error)
r, _, _ = select.select([s], [], [])
if r:
    data = s.recv(4096)
    if data == b"":
        print("server closed")
```

`select` with no timeout blocks indefinitely — this is the "wait forever" pattern. Add a timeout for safety:

```python
r, _, _ = select.select([s], [], [], timeout=30)
if not r:
    print("no close within 30s")
```

4. Check for a *half-close* — server stopped sending but still reading

If the server did `shutdown(SHUT_WR)` (half-close) rather than full `close()`, you'll see EOF on `recv()` but your `send()` still works. That's often intentional (request/response protocols). Don't assume EOF means the whole connection is dead — try sending and see.

5. Don't rely on these to detect close

- × `sock.getpeername()` — doesn't reliably raise on FIN; behavior varies.
- × Checking `sock.fileno() == -1` — only true after *you* `close()`.
- × A `send()` success — TCP buffers locally; a send can "succeed" after the peer

- ✗ `send()` success — TCP buffers locally, a send can succeed after the peer closed.
- ✗ `sock.recv()` with `MSG_PEEK` repeatedly — works but is a busy loop unless combined with `select`.

6. Timeouts / non-blocking, if you can't block forever

```python
s.settimeout(30)          # applies to recv/send/connect
try:
    while True:
        data = s.recv(4096)
        if not data:
            print("closed")
            break
except socket.timeout:
    print("gave up waiting")
```

Or non-blocking + `selectors` for multiplexing many sockets:

```python
import selectors, socket

sel = selectors.DefaultSelector()
s = socket.socket()
s.connect(("localhost", 12345))
s.setblocking(False)
sel.register(s, selectors.EVENT_READ)

while True:
    for key, _ in sel.select(timeout=30):   # blocks here
        data = key.fileobj.recv(4096)
        if not data:
            print("server closed")
            sel.unregister(s)
            s.close()
            raise SystemExit
```

7. The tricky case: server vanishes without FIN (network drop)

If the peer crashes or the network dies, no FIN/RST arrives. `recv()` will block forever. Only these help:

- `settimeout()` / `select(timeout=...)` — your own inactivity timeout.
- `SO_KEEPALIVE` — kernel probes idle connections (defaults ~2 hours; tune with `TCP_KEEPIDLE` / `TCP_KEEPINTVL` / `TCP_KEEPCNT` on Linux):

```python
s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIDLE, 60)
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPINTVL, 10)
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT, 3)
```

- **Application-level heartbeat/ping** — most portable.

Summary table

Situation	Detection
Graceful close (FIN)	`recv()` → `b''`
Abortive close (RST)	`recv()` / `send()` → `ConnectionResetError`
Write after peer closed	`send()` → `BrokenPipeError` (eventually)
Peer vanished, no FIN	Only timeouts / `SO_KEEPALIVE` / app heartbeat
Want to *block* until close	`select([s], [], [])` then `recv()`, or just `recv()`

The single most important rule: **always end up calling `s.close()` yourself after you detect EOF** — the OS will not do it for you.

Shortest answer, what is a connection reset error?

`ConnectionResetError` = the peer (or network) sent a **TCP RST**, abruptly killing the connection instead of a graceful FIN. Your next `recv()` / `send()` raises it. Common causes: server crashed, closed with unread data, or `SO_LINGER=0`.

Shortest answer, only code, set timeout on recv

Shortest answer, only code, set timeout on recv

```python
s.settimeout(5)  # seconds
try:
    data = s.recv(4096)
except socket.timeout:
    pass
```

> And if the system shuts down with a SystemExit, will the remaining resources be closed?

No. `SystemExit` unwinds the stack like any exception — `try/finally` and `with` blocks **do** run, so resources closed there are released. But anything relying only on the process ending (unclosed sockets, files) is left to the OS, which reclaims fds on process exit — so the kernel cleans up, but your code's cleanup (flushing buffers, sending FIN gracefully, deleting temp files) does **not** run unless you wrapped it in `finally` / `with` .

> So the socket remains open while th einterpreter is shutdown?

Yes. During `SystemExit` unwinding (and interpreter shutdown), the socket fd stays open — no `close()` , no FIN sent by your code — until the process actually exits. At exit the kernel closes the fd, which then sends FIN (or RST). So: open throughout shutdown, closed only by the OS at process termination.

> But I am unittesting, so what to do

In a unit test, don't rely on process exit — close explicitly so tests don't leak fds across runs (same process!).

```python
def test_foo(self):
    s = socket.socket()
    try:
        s.connect(...)
        ...
    finally:
        s.close()
```

Or use `addCleanup` so it runs even if the test fails:

```python
def test_foo(self):
    s = socket.socket()
    self.addCleanup(s.close)
    ...
```

Or `with contextlib.closing(socket.socket()) as s:` .

Also call `s.shutdown(socket.SHUT_RDWR)` before `close()` if you want to force FIN/RST deterministically and avoid TIME_WAIT surprises between tests.

And if the class has a tearDown will it always execute like a finally?

Yes — `tearDown` runs even if the test fails or raises (`SystemExit` included), as long as the test method was entered. It behaves like a `finally` around the test.

Caveats:

- If `setUp` raises, `tearDown` is **not** called.
- `SystemExit` / `KeyboardInterrupt` in the test: `tearDown` still runs under normal `unittest` — but if the whole process is killed (`os._exit` , SIGKILL) nothing runs.
- `addCleanup` is more robust than `tearDown` (LIFO, runs even if `setUp` partially succeeded).

Will a self.fail() trigger self.tearDown() ?

Yes. `self.fail()` raises `AssertionError` , which `unittest` catches as a normal test failure — `tearDown` still runs afterward. Same as any exception in the test body.

how to prove if socket is open

```python
s.fileno() != -1        # fd still allocated (does NOT mean peer is alive)
```

That only tells you *you* haven't closed it. It says nothing about the peer. To prove the

connection is still usable, you must touch the network:

```python
# non-destructive check: is peer still there?
import select
r, _, _ = select.select([s], [], [], 0)
if r:
    try:
        data = s.recv(1, socket.MSG_PEEK)   # don't consume
    except ConnectionResetError:
        open_ = False
    else:
        open_ = data != b""                 # b'' = EOF = peer closed
else:
    open_ = True                            # nothing pending; assume alive (can't pro
ve)
```

There is **no** way to prove a TCP connection is alive without sending/receiving — a silent peer looks identical to a dead one. Only `send()` + `recv()` (or a heartbeat) gives real proof.

How to place a sttring inside bytes()

```python
b = bytes("hello", "utf-8")     # or
b = "hello".encode("utf-8")
```

Not `bytes("hello")` — that raises `TypeError` (needs an encoding).

More like bytes([STX, string, ETX])

```python
b = bytes([0x02]) + string.encode("utf-8") + bytes([0x03])
```

`bytes([...])` takes ints (0–255), not strings — so encode the string part and concatenate. Or:

```python
b = b"".join([bytes([0x02]), string.encode(), bytes([0x03])])
```

Not like?:
bytes([WS_REGISTER, 3, 50, 50, STX, *list("Six Seven"), ETX, AVAILABLE])

That works — `bytes([...])` accepts a mix of ints and iterables of ints, and `*list("Six Seven")` unpacks the string into individual **characters**, which are *not* ints. So it fails:

```python
bytes([... , *list("Six Seven"), ...])
# TypeError: 'str' object cannot be interpreted as an integer
```

Fix: use the character codes, e.g. `*"Six Seven".encode()`, or `*map(ord, "Six Seven")`:

```python
bytes([WS_REGISTER, 3, 50, 50, STX, *"Six Seven".encode(), ETX, AVAILABLE])
```

(Note: `ord("S")` happens to be 83, not 50 — `50` as an int is the ASCII digit `'2'`. If you meant the *text* "50", use `*b"50"`.)

But then pyright says unnecesary call to encode

Pyright is right — `encode()` is redundant when you're already splatting into `bytes([...])`? No: the splat needs ints, `encode()` gives a `bytes`, and splatting a `bytes` yields ints. So it *is* needed here.

But pyright likely flags it because it infers the literal is ASCII and suggests `b"Six Seven"`. Use a bytes literal directly:

```python
bytes([WS_REGISTER, 3, 50, 50, STX, *b"Six Seven", ETX, AVAILABLE])
```

Splatting a `bytes` literal yields ints, no `encode()` call, no warning. If the string is dynamic and non-ASCII, keep `.encode()` and ignore/suppress the warning — it's a false positive for your use case.